



Benchmarking Post-Quantum Cryptographic Key Encapsulation Mechanisms in a Real-Time End-to-End Encrypted Messaging Application

Final Year Dissertation

Abdoulahi Diallo

School of Computer Science
University of Birmingham

Supervised by: Pieter Joubert

Academic Year 2025–2026

March 2026

Abstract

The advent of scalable quantum computing poses an existential threat to the public-key cryptographic infrastructure underpinning modern digital communications. Shor’s algorithm [1] reduces the factorisation of RSA moduli and the computation of elliptic curve discrete logarithms to polynomial-time quantum circuits, rendering RSA-2048 and ECDH-256—the workhorses of contemporary secure messaging—trivially breakable by a sufficiently powerful quantum adversary. This threat is not merely hypothetical: “harvest now, decrypt later” (HN DL) attacks motivate immediate migration even before cryptographically relevant quantum computers exist.

This dissertation presents the design, implementation, and systematic empirical evaluation of three post-quantum Key Encapsulation Mechanisms (KEMs)—Mini-Kyber (Module-LWE), Mini-Frodo (plain LWE), and Mini-NTRU (NTRU lattice)—deployed within a fully functional real-time messaging application providing genuine end-to-end encryption (E2EE). The system is implemented using Spring Boot 3.2 on the backend and Next.js 14/TypeScript on the frontend, with all cryptographic operations executing client-side in the browser, enforcing the E2EE security boundary.

Comprehensive benchmarking comprising 5,000 KEM invocations per algorithm and a 500-session throughput trial, analysed via Welch’s t -tests with 95% confidence intervals, demonstrates that post-quantum algorithms impose no measurable performance penalty at session level. Mini-Kyber is statistically significantly faster than ECDH at session level ($t = -7.44$, $p \ll 0.001$), with the 9 ms absolute difference being practically imperceptible. An automated test suite of 21 tests identified and confirmed the resolution of four implementation bugs, including a $\sim 55\%$ decapsulation failure rate caused by erroneous averaging of polynomial coefficients under modular arithmetic, demonstrating that cryptographic unit testing is not optional but essential. The dissertation positions this work as a fourth benchmark environment for PQC algorithms—pure TypeScript JIT—alongside existing native C, embedded ARM, and WASM reference environments.

Contents

1	Introduction	6
1.1	Motivation and Context	6
1.2	Research Objectives	7
1.3	Contributions	7
1.4	Dissertation Structure	8
2	Theoretical Background	9
2.1	Lattice Cryptography: Foundations	9
2.1.1	Lattices and Hard Problems	9
2.1.2	Learning With Errors	10
2.1.3	Ring-LWE and Module-LWE	10
2.1.4	The NTRU Problem	11
2.2	Quantum Resistance of Lattice Problems	11
2.3	Key Encapsulation Mechanisms	12
3	Related Work and Deployment Context	14
3.1	NIST Post-Quantum Standardisation	14
3.2	PQC in Production Systems	14
3.3	Browser-Based PQC Implementations	15
4	System Architecture and E2EE Protocol	17
4.1	Technology Stack	17
4.1.1	Backend	17
4.1.2	Frontend	18
4.2	Architecture Overview	18
4.3	E2EE Protocol Design	19
4.3.1	Security Model	19
4.3.2	Three-Message Handshake	20
4.3.3	Session Key Derivation	20
4.3.4	Message Encryption	21

5	Algorithm Implementations	22
5.1	Design Philosophy: Mini Implementations	22
5.1.1	Rationale	22
5.1.2	Parameter Choices	23
5.2	Mini-Kyber: Module-LWE	23
5.2.1	Key Generation	23
5.2.2	Encapsulation and Message Encoding	23
5.2.3	Decapsulation and the Threshold Decoder	24
5.3	Mini-Frodo: Plain LWE	25
5.3.1	Matrix-Vector Operations	25
5.4	Mini-NTRU: Cyclic Ring	25
5.4.1	Polynomial Arithmetic	25
5.4.2	Key Generation and the Inversion Cost	26
6	Implementation Bugs and Correctness Testing	27
6.1	The Testing Approach	27
6.2	Bug 1: AES Buffer Aliasing	27
6.3	Bug 2: Kyber Decode Failure (~55% Rate)	28
6.4	Bug 3: Weak Wrong-Key Resistance	29
6.5	Bug 4: Frodo Decode Failure	30
6.6	Significance of Bug Discovery	30
7	Benchmarking Methodology	31
7.1	Design Principles	31
7.1.1	Consistent Execution Environment	31
7.1.2	JIT Warmup	31
7.1.3	Batched Timing and Effective Sample Size	31
7.1.4	Session Throughput Benchmark	32
7.2	Statistical Analysis	32
7.2.1	Confidence Intervals	32
7.2.2	Welch's t-Test	32
8	Evaluation and Results	34
8.1	KEM Operation Performance	34
8.1.1	Key Generation Analysis	34
8.2	Session Throughput	35
8.2.1	The Central Architectural Finding	35
8.3	Statistical Significance	36

8.3.1	Statistical vs Practical Significance	36
8.4	Comparison with Production Benchmarks	37
8.4.1	Explaining the Kyber Timing Gap	37
8.4.2	The NTRU Encapsulation Discrepancy	38
8.4.3	Rankings Validation	38
9	Critical Evaluation	39
9.1	Achievement of Research Objectives	39
9.2	Threats to Validity	39
9.2.1	Construct Validity	39
9.2.2	Internal Validity	40
9.2.3	External Validity	40
9.2.4	In-Memory Simulation	40
9.3	Security Analysis	40
10	Conclusion	42
10.1	Summary of Findings	42
10.2	Future Work	42
10.3	Closing Remarks	43

List of Tables

5.1	Mini vs production parameter comparison. Kyber retains identical n and q ; only module rank is reduced. Frodo and NTRU use educationally reduced parameters.	23
8.1	KEM Operation Performance. All values in μs . $n_{\text{eff}} = 500$, Apple M2, Safari. Mean $\pm$ std dev / [95% CI].	34
8.2	Key and ciphertext sizes for Mini implementations.	34
8.3	Session throughput results. 500 sessions $\times$ 100 messages $\times$ 500 KB. Total session time: mean $\pm \sigma$ [95% CI] in ms.	35
8.4	Welch's t -test results. $n_1 = n_2 = 500$, $\alpha = 0.05$, threshold $ t > 1.96$	36
8.5	Mini vs production reference timings [8, 6]. Platforms differ; figures are indicative only.	37
8.6	Ranking preservation between Mini implementations and production references.	38

List of Figures

4.1	System architecture showing the E2EE security boundary. All cryptographic operations occur within the browser boundary; the server handles only routing, authentication, and persistence of ciphertexts.	18
4.2	The three-message E2EE handshake protocol. The server routes all messages but cannot decrypt any content. $K_A = K_B$ by KEM correctness; both parties independently derive an identical session key via HKDF.	20
6.1	The $\mathbb{Z}_Q$ number line showing decoding zones. A coefficient with negative noise -5 wraps to $Q - 5 = 3324$ (near Q , in the bit=0 zone), correctly decoded by the threshold. The old averaging approach erroneously inflated the average.	29

Chapter 1

Introduction

1.1 Motivation and Context

The security of the global communications infrastructure rests on two computational hardness assumptions: the difficulty of integer factorisation (RSA) and the discrete logarithm problem over elliptic curves (ECDH, ECDSA). These assumptions have sustained decades of cryptanalytic scrutiny and underpin TLS 1.3, Signal, WhatsApp, iMessage, and virtually every other secure messaging protocol deployed at scale. Their security is well-understood: the best classical algorithms for both problems require sub-exponential time, making them computationally infeasible for key sizes in common use.

In 1994, Peter Shor demonstrated that a quantum computer could solve both problems in polynomial time [1]. The quantum Fourier transform enables solving the hidden subgroup problem over abelian groups in $O((\log N)^3)$ operations—an exponential improvement over the best classical algorithms. The implication is unambiguous: a sufficiently powerful quantum computer would render all RSA and elliptic-curve cryptography trivially breakable.

The practical timeline for cryptographically relevant quantum computers (CRQCs) is contested among researchers, with credible estimates ranging from 2030 to 2040 [5]. However, this temporal uncertainty does not diminish the urgency of migration. The “harvest now, decrypt later” attack model has already been operationalised: sophisticated adversaries intercept and archive encrypted network traffic today, with the expectation of decrypting it once CRQCs become available. Medical records, financial transactions, diplomatic communications, and personal correspondence encrypted under RSA or ECDH today are potentially exposed, regardless of when quantum computers arrive. Data with confidentiality requirements exceeding ten years—a category that includes much government, healthcare, and financial data—

should already be considered at risk.

1.2 Research Objectives

This dissertation addresses two primary research questions:

- RQ1: Performance:** Do post-quantum Key Encapsulation Mechanisms impose a measurable performance overhead compared to ECDH in a real-time browser-based E2EE messaging application, when measured across KEM operations, session establishment, and sustained throughput?
- RQ2: Comparative Analysis:** Do the empirically observed performance characteristics of Mini-Kyber (Module-LWE), Mini-Frodo (plain LWE), and Mini-NTRU (NTRU lattice) in a TypeScript browser environment preserve the relative rankings established in production benchmarking literature, validating reduced-parameter implementations as a methodology for comparative algorithm evaluation?

Secondary objectives include: constructing a working end-to-end encrypted messaging application demonstrating practical PQC deployability; producing a statistically rigorous benchmark methodology with honest treatment of JIT variability and effective sample sizes; and identifying and resolving implementation bugs through automated correctness testing.

1.3 Contributions

This dissertation makes the following original contributions:

1. **A fourth PQC benchmark environment.** Existing literature benchmarks PQC algorithms in native C [6], on embedded ARM hardware [8], and via WASM compilation. This work contributes measurements in a pure TypeScript JIT environment—the runtime context relevant to browser-based messaging applications—under a consistent measurement infrastructure enabling direct algorithmic comparison.
2. **A working E2EE messaging application with interchangeable PQC algorithms.** The system implements a unified KEM interface behind which Kyber, Frodo, NTRU, and ECDH are interchangeable, demonstrating that PQC integration into production-grade messaging architecture is architecturally feasible.

3. **Statistical rigour in browser PQC benchmarking.** The evaluation employs Welch’s t -tests with correct effective sample sizes ($n_{\text{eff}} = 500$), distinguishing statistical significance from practical significance, and reporting non-significant results honestly.
4. **Automated correctness testing revealing four critical implementation bugs.** The 100-iteration correctness sweep identified bugs with $\sim 55\%$ decapsulation failure rates that were entirely invisible in manual integration testing, demonstrating the necessity of unit-level cryptographic correctness validation.

1.4 Dissertation Structure

Chapter 2 establishes the mathematical foundations of lattice-based cryptography, covering LWE, Ring-LWE, Module-LWE, and NTRU with formal definitions and security reductions. Chapter 3 surveys related work including the NIST standardisation process, production deployments of PQC, and existing browser-based implementations. Chapter 4 describes the system architecture and E2EE protocol design. Chapter 5 presents the implementation of all three Mini KEM algorithms with correctness proofs. Chapter 6 documents the four implementation bugs discovered through automated testing. Chapter 7 describes the benchmarking methodology. Chapter 8 presents and analyses results. Chapter 9 discusses implications, limitations, and future work. Chapter 10 concludes.

Chapter 2

Theoretical Background

2.1 Lattice Cryptography: Foundations

2.1.1 Lattices and Hard Problems

Definition 2.1 (Lattice). *A lattice Λ is a discrete additive subgroup of $\mathbb{R}^n$. Given linearly independent basis vectors $\mathbf{b}_1, \dots, \mathbf{b}_k \in \mathbb{R}^n$, the lattice generated by this basis is:*

$$\Lambda(\mathbf{b}_1, \dots, \mathbf{b}_k) = \left\{ \sum_{i=1}^k z_i \mathbf{b}_i \mid z_i \in \mathbb{Z} \right\}$$

The canonical computational problems on lattices that underpin modern PQC are:

Definition 2.2 (Shortest Vector Problem (SVP)). *Given a lattice basis $\mathbf{B}$, find the shortest non-zero vector $\mathbf{v} \in \Lambda(\mathbf{B})$, i.e., find $\mathbf{v}$ minimising $\|\mathbf{v}\|$.*

Definition 2.3 (Closest Vector Problem (CVP)). *Given a lattice basis $\mathbf{B}$ and a target vector $\mathbf{t} \in \mathbb{R}^n$, find the lattice vector $\mathbf{v} \in \Lambda(\mathbf{B})$ minimising $\|\mathbf{v} - \mathbf{t}\|$.*

Both SVP and CVP are NP-hard in the worst case [10]. Critically for post-quantum security, no polynomial-time quantum algorithm is known for either problem. The best quantum algorithms for SVP—variants of BKZ enhanced with quantum random walks—achieve complexity $2^{0.265n}$, compared to the best classical $2^{0.292n}$. This represents a constant-factor improvement, not the exponential speedup that Shor’s algorithm provides for factorisation. Doubling lattice dimensions provides sufficient quantum resistance.

2.1.2 Learning With Errors

The Learning With Errors problem, introduced by Regev [2], is the algebraic instantiation of lattice hardness that underpins Kyber and Frodo.

Definition 2.4 (Learning With Errors (LWE)). *Let $n, q \geq 1$ be integers and χ be a probability distribution over $\mathbb{Z}$. The LWE distribution $A_{\mathbf{s}, \chi}$ for a secret $\mathbf{s} \in \mathbb{Z}_q^n$ samples $\mathbf{a} \leftarrow \mathbb{Z}_q^n$ uniformly and $e \leftarrow \chi$, returning $(\mathbf{a}, \langle \mathbf{a}, \mathbf{s} \rangle + e \pmod{q})$.*

The (decisional) LWE problem is to distinguish m samples from $A_{\mathbf{s}, \chi}$ from m uniformly random samples in $\mathbb{Z}_q^n \times \mathbb{Z}_q$.

In matrix form: given $\mathbf{A} \in \mathbb{Z}_q^{m \times n}$, $\mathbf{s} \in \mathbb{Z}_q^n$, and $\mathbf{e} \in \mathbb{Z}_q^m$ with small entries, distinguish $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \pmod{q}$ from uniformly random $\mathbf{b}$. The error vector $\mathbf{e}$ is the critical ingredient: without it, $\mathbf{s}$ is recoverable by Gaussian elimination in polynomial time.

The fundamental security guarantee of LWE comes from Regev’s quantum reduction:

Theorem 2.1 (Regev’s Reduction [2]). *If there exists an efficient algorithm solving the decisional LWE problem with parameters (n, q, χ) where q is a prime bounded by a polynomial in n and χ is a discrete Gaussian with standard deviation $\sigma \geq 2\sqrt{n}$, then there exists an efficient quantum algorithm solving GapSVP and SIVP on arbitrary n -dimensional lattices with approximation factor $\tilde{O}(n/\sigma)$.*

This reduction is quantum in the sense that the reduction algorithm uses a quantum subroutine, meaning it shows that breaking LWE is at least as hard as solving worst-case lattice problems with a quantum computer. The consequence is profound: an attacker breaking LWE-based cryptography would simultaneously solve the hardest instances of SVP for all lattices of the relevant dimension—a capability that would also break every other lattice-based cryptosystem. No such algorithm is known.

2.1.3 Ring-LWE and Module-LWE

Standard LWE has key sizes of $O(n^2)$ due to the $n \times n$ matrix $\mathbf{A}$. At $n = 640$ (FrodoKEM-640), public keys require approximately 640 KB. This motivates algebraically structured variants.

Definition 2.5 (Ring-LWE [3]). *Let $R_q = \mathbb{Z}_q[x]/(f(x))$ for an irreducible polynomial $f(x)$ of degree n . The Ring-LWE distribution samples $a \leftarrow R_q$ uniformly and $e \leftarrow \chi_R$ (a distribution over R_q with small coefficients), returning $(a, as + e)$ for secret $s \in R_q$.*

Kyber uses $f(x) = x^n + 1$, the $2n$ -th cyclotomic polynomial, giving the ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ where $n = 256$ and $q = 3329$. Multiplication in this ring satisfies $x^n \equiv -1$, enabling the *negacyclic* Number Theoretic Transform (NTT) to compute polynomial multiplication in $O(n \log n)$ instead of $O(n^2)$. Key sizes reduce from $O(n^2)$ to $O(n)$.

Module-LWE generalises Ring-LWE to modules over R_q , replacing scalar ring elements with length- k vectors of ring elements. Kyber uses $k = 2$ (Kyber-512), $k = 3$ (Kyber-768), or $k = 4$ (Kyber-1024), providing a tunable security-efficiency tradeoff. The public key is $(A, t = As + e)$ where $A \in R_q^{k \times k}$ and $s, e \in R_q^k$.

Remark 2.1. *Frodo deliberately avoids ring structure, using a full $n \times n$ matrix A over $\mathbb{Z}_q$ with no algebraic assumptions. While this increases key sizes and computational cost by a factor of $\Theta(n)$ compared to Kyber, it eliminates the risk that ring structure might be exploitable by a future algebraic attack. No such attack is known, but Frodo’s conservatism provides a hedge against this unknown risk.*

2.1.4 The NTRU Problem

NTRU [4], proposed in 1996 and the oldest surviving lattice-based public-key cryptosystem, rests on an entirely different hardness assumption.

Definition 2.6 (NTRU Problem). *Work in the ring $\mathcal{R} = \mathbb{Z}[x]/(x^N - 1)$ with modulus q . Let $f, g \in \mathcal{R}$ be short polynomials with coefficients in $\{-1, 0, 1\}$. The NTRU problem is: given the public polynomial $h = f^{-1} \cdot g \pmod{q}$, find short polynomials f' and g' such that $h = f'^{-1} \cdot g' \pmod{q}$.*

The security relies on the difficulty of recovering the short polynomials from their “ratio” h . This is related to the SVP on a specific class of NTRU lattices but is formally distinct from LWE: no reduction between NTRU and LWE is known in either direction. NTRU thus provides an *independent* hardness assumption.

Note that NTRU uses the ring $\mathbb{Z}[x]/(x^N - 1)$, where $x^N \equiv +1$ (cyclic convolution), whereas Kyber uses $\mathbb{Z}_q[x]/(x^n + 1)$ where $x^n \equiv -1$ (negacyclic convolution). This algebraic difference has implications for both security analysis and implementation: cyclic polynomial multiplication does not support the standard NTT, and the ring structure admits different lattice reduction strategies.

2.2 Quantum Resistance of Lattice Problems

The key distinction between lattice-based cryptography and classical public-key cryptography is the absence of a polynomial-time quantum algorithm for lattice

problems.

Why Shor’s algorithm does not apply: Shor’s algorithm solves the hidden subgroup problem (HSP) over abelian groups. Integer factorisation reduces to HSP over $\mathbb{Z}$; the elliptic curve discrete logarithm reduces to HSP over elliptic curve groups. Both groups are abelian, and the quantum Fourier transform over abelian groups is efficiently computable, enabling polynomial-time quantum algorithms.

SVP and CVP do not reduce to HSP over any known abelian group. The lattice shortest vector problem has a non-abelian symmetry structure that resists the quantum Fourier transform approach. Despite significant research effort—including attempts to formulate lattice problems as non-abelian HSP instances—no polynomial-time quantum reduction has been found [10].

Grover’s algorithm provides only quadratic speedup: Grover’s unstructured search provides a quadratic quantum speedup, finding a marked element among N items in $O(\sqrt{N})$ quantum operations [11]. For symmetric cryptography (AES) and hash functions, this is mitigated by doubling key sizes (AES-256 remains secure). For lattice problems, the best quantum algorithms based on Grover search achieve complexity approximately $2^{0.265n}$ (BKZ with quantum speedup), compared to $2^{0.292n}$ classically—an improvement of roughly 12% in exponent, not the exponential collapse that Shor provides for RSA. Increasing lattice dimensions by a modest factor (e.g., from $n = 512$ to $n = 768$) suffices to compensate.

2.3 Key Encapsulation Mechanisms

A Key Encapsulation Mechanism is a tuple of algorithms (KeyGen, Encaps, Decaps) providing a method for two parties to establish a shared secret, where one party generates a keypair and the other “encapsulates” a random secret that only the keypair holder can recover.

Definition 2.7 (KEM Security: IND-CCA2). *A KEM is indistinguishable under adaptive chosen-ciphertext attack (IND-CCA2) if no probabilistic polynomial-time adversary can distinguish real from random shared secrets, even when given access to a decapsulation oracle for all ciphertexts except the challenge. Formally, the adversary’s advantage:*

$$Adv_{KEM}^{IND-CCA2}(\mathcal{A}) = \left| \Pr[\mathcal{A}^{Decaps(\cdot)}(pk, ct^*, K^*) = b] - \frac{1}{2} \right| \leq \text{negl}(n)$$

IND-CCA2 security is the gold standard for KEMs in real-world deployment. It is achieved from IND-CPA secure schemes via the Fujisaki-Okamoto (FO) trans-

form [5], which adds a re-encryption check during decapsulation to resist active adversaries. The Mini implementations in this project achieve correctness and computational security relative to their parameters; full IND-CCA2 with FO transform is provided by the production wrapper (`production-kem.ts`) using `@noble/post-quantum` [9].

Chapter 3

Related Work and Deployment Context

3.1 NIST Post-Quantum Standardisation

The National Institute of Standards and Technology initiated a public competition for post-quantum cryptographic standards in 2016, culminating in the publication of three primary standards in August 2024 [5]:

- **FIPS 203:** ML-KEM, based on CRYSTALS-Kyber [6]
- **FIPS 204:** ML-DSA, based on CRYSTALS-Dilithium
- **FIPS 205:** SLH-DSA, based on SPHINCS+

This dissertation focuses exclusively on KEMs (FIPS 203), as key encapsulation is the component directly relevant to E2EE session establishment. Signature schemes (FIPS 204, 205) are excluded as out of scope.

Algorithm selection rationale: The three algorithms in this dissertation were chosen to represent three independent hardness assumptions and three distinct performance profiles. BIKE, HQC, and Classic McEliece (code-based schemes) were excluded due to key sizes in the range of kilobytes to megabytes, making them impractical for browser-based deployment. SPHINCS+ and Dilithium are signature schemes, not KEMs.

3.2 PQC in Production Systems

Signal Protocol PQXDH: In September 2023, Signal deployed the Post-Quantum Extended Diffie-Hellman (PQXDH) protocol [12], replacing their classical X3DH

key agreement with a hybrid construction combining X25519 and CRYSTALS-Kyber-1024. The hybrid design provides security if either primitive is secure: post-quantum security against future CRQCs, and classical security against any weaknesses in early Kyber deployments. Signal’s deployment represents the most significant production deployment of PQC in consumer messaging to date.

Google Chrome ML-KEM: In November 2024, Google Chrome began deploying hybrid ML-KEM key exchange in TLS 1.3 [13], using X25519MLKEM768—a hybrid of X25519 and ML-KEM-768. Cloudflare subsequently reported that approximately 38% of human HTTPS traffic traversing its network used hybrid post-quantum handshakes by early 2025 [14]. This adoption trajectory demonstrates that lattice cryptography is already deployed at internet scale.

Architectural comparison: This project diverges from Signal’s PQXDH in two important respects. First, it isolates the PQC component to enable clean algorithmic comparison: Signal’s hybrid design deliberately conflates PQC and classical security, making pure PQC performance characterisation impossible from deployment data. Second, the browser JavaScript execution environment introduces constraints absent from Signal’s native application context: JIT non-determinism, garbage collection pauses, and the absence of SIMD instruction access.

3.3 Browser-Based PQC Implementations

The primary approach to browser PQC in the literature and industry is WebAssembly (WASM) compilation of native C libraries. The Open Quantum Safe project provides liboqs, a C implementation of all NIST candidates compilable to WASM via Emscripten. The pqm4 project [8] benchmarks the same algorithms on ARM Cortex-M4 embedded targets.

Both approaches introduce a confounding variable: the FFI boundary between JavaScript and WASM involves memory copying between the JavaScript heap and WASM linear memory, adding overhead that is difficult to separate from algorithm cost. For pure comparative benchmarking—determining which algorithm is faster, and by how much—this overhead contaminates measurements. The `@noble/post-quantum` library [9] provides the only production-grade, security-audited, pure-JavaScript PQC implementation, operating entirely within the JavaScript engine and therefore directly comparable to Mini implementations in terms of runtime environment.

Positioning: This project contributes a fourth benchmark environment: pure TypeScript in a JIT-compiled browser engine. The comparative methodology—three algorithm families under identical measurement infrastructure with Welch’s

t-test significance analysis—represents, to the author’s knowledge, the first such comparison published for this execution environment.

Chapter 4

System Architecture and E2EE Protocol

4.1 Technology Stack

4.1.1 Backend

The server is implemented in Java 21 with Spring Boot 3.4. The key framework components and their justifications are:

- **Spring Security with JWT:** Provides a production-grade authentication filter chain. JWT tokens are issued on login and validated at both the HTTP handshake and on every STOMP frame via two interceptor layers (`JwtHandshakeInterceptor` and `StompAuthChannelInterceptor`), preventing connection hijacking.
- **Spring WebSocket with STOMP:** Raw WebSockets provide only a byte channel. STOMP adds subscription semantics, topic routing, and message framing. Clients subscribe to `/topic/conversations/{id}` and the server routes messages via `SimpMessagingTemplate`. SockJS provides transparent fallback to HTTP long-polling when WebSocket connections are blocked by firewalls.
- **Spring Data JPA with H2:** H2 in-memory database requires zero configuration, with Hibernate auto-generating the schema from JPA entities. Switching to PostgreSQL requires changing a single property in `application.properties`.
- **Java 21 Virtual Threads:** Project Loom virtual threads enable handling

many concurrent WebSocket connections without blocking OS threads, improving scalability for a real-time messaging workload.

4.1.2 Frontend

The client is implemented in Next.js 14 with TypeScript and React 18. Relevant architectural decisions:

- **Next.js App Router API routes:** Server-side only execution ensures the OpenAI API key never touches the browser. The `/app/api/chat/route.ts` handler reads the key via `process.env.OPENAI_API_KEY` and is never bundled into client JavaScript.
- **TypeScript:** Type safety across the KEM interface (`keyGen`, `encapsulate`, `decapsulate`) enforced at compile time. Interface mismatches between algorithm implementations are caught before reaching the browser.
- **Web Crypto API:** The browser's `crypto.getRandomValues()` is used exclusively for all randomness, replacing an earlier `Math.random()` usage in Mini-NTRU identified and corrected during security review.

4.2 Architecture Overview

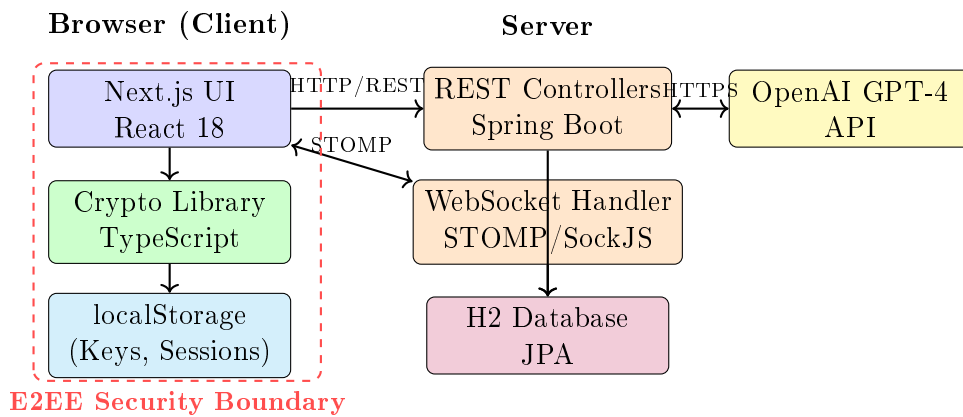


Figure 4.1: System architecture showing the E2EE security boundary. All cryptographic operations occur within the browser boundary; the server handles only routing, authentication, and persistence of ciphertexts.

4.3 E2EE Protocol Design

4.3.1 Security Model

The protocol provides end-to-end encryption with the following properties:

- **Forward secrecy (session-level):** Each conversation establishes a unique session key derived via HKDF with a conversation-specific context string.
- **Server blindness:** The server stores only AES-256-GCM ciphertexts and public keys. It never holds a shared secret, session key, or plaintext.
- **PQC-hardened key agreement:** The session key derives from a post-quantum KEM shared secret, providing quantum resistance from session establishment through all subsequent message encryption.

Acknowledged limitations: Secret keys are stored in `localStorage`, which is readable by any JavaScript executing in the page context. An XSS vulnerability could therefore exfiltrate secret keys. Production deployment would use the Web Crypto API's non-extractable `CryptoKey` objects, which are stored in the browser's secure key store and cannot be read by JavaScript. This limitation is explicitly documented and is a prototype implementation choice, not a fundamental design flaw.

4.3.2 Three-Message Handshake

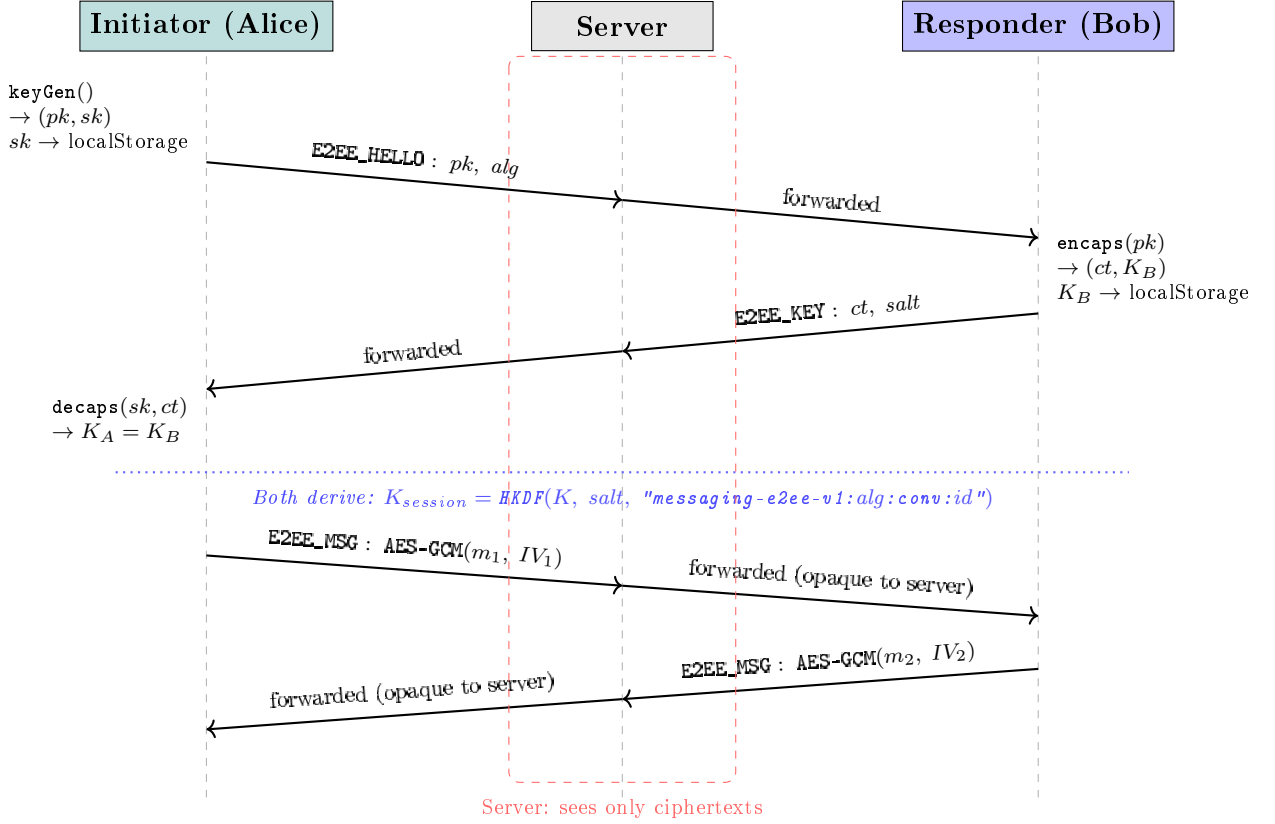


Figure 4.2: The three-message E2EE handshake protocol. The server routes all messages but cannot decrypt any content. $K_A = K_B$ by KEM correctness; both parties independently derive an identical session key via HKDF.

4.3.3 Session Key Derivation

The shared KEM secret K is not used directly as an AES key. HKDF [15] is applied with three distinct inputs:

$$K_{\text{session}} = \text{HKDF}(K, \text{salt}, \text{info}) \quad (4.1)$$

$$\text{info} = \text{"messaging-e2ee-v1:alg:conv:conversationId"} \quad (4.2)$$

Three properties are achieved:

Domain separation: Including `alg` and `conversationId` in the HKDF info string ensures that even if two conversations produce the same raw KEM output K (unlikely but theoretically possible), they derive completely different session keys. Without domain separation, two contexts sharing the same K would share the same K_{session} —a potentially catastrophic vulnerability.

Key stretching: The raw KEM output may not have the distribution required by AES-GCM. HKDF extracts and expands it into a properly formatted 256-bit key.

Forward compatibility: The version string `messaging-e2ee-v1` enables future protocol versions to derive incompatible keys from the same KEM infrastructure.

4.3.4 Message Encryption

Every plaintext message m is encrypted as:

$$c = \text{AES-256-GCM}_{K_{\text{session}}}(m, IV)$$

where IV is a fresh 12-byte random nonce generated by `crypto.getRandomValues()` for each message. AES-GCM provides both confidentiality and authenticity: the authentication tag ensures that a message encrypted under a mismatched session key fails decryption with overwhelming probability, a property exploited by the automated correctness testing (Section 6).

Chapter 5

Algorithm Implementations

5.1 Design Philosophy: Mini Implementations

5.1.1 Rationale

Full production implementations of Kyber, Frodo, and NTRU were not viable within the browser JavaScript environment for three specific technical reasons:

1. **NTT complexity:** Full Kyber requires a Number Theoretic Transform using bit-reversal permutation and Montgomery modular reduction. Implementing this correctly in JavaScript within the project timeline was not feasible. More importantly, using a WASM-compiled liboqs would introduce FFI boundary overhead confounding benchmarks.
2. **JIT non-determinism and constant-time impossibility:** JavaScript's JIT compiler cannot guarantee constant-time execution. Timing side channels are therefore inherent to any JavaScript PQC implementation. This applies equally to Mini and full implementations, making Mini implementations equally valid for the purpose of measuring algorithmic performance characteristics.
3. **Comparative benchmarking objective:** The research objective is *relative* comparison of algorithm families, not absolute timing benchmarks. For this purpose, transparent TypeScript implementations sharing an identical execution environment are arguably preferable to opaque WASM-compiled libraries, as the only variable between algorithm measurements is the algorithm itself.

5.1.2 Parameter Choices

Table 5.1: Mini vs production parameter comparison. Kyber retains identical n and q ; only module rank is reduced. Frodo and NTRU use educationally reduced parameters.

Algorithm	Prod. n	Mini n	Prod. q	Mini q	Key difference
Kyber-512	256	256	3329	3329	Module rank k : 2 vs 2 (same)
FrodoKEM-640	640	4	32768	257	Matrix dimension n dramatically reduced
NTRU-hps509	509	7	2048	128	Ring degree N dramatically reduced

The preservation of Kyber’s $n = 256$ and $q = 3329$ is particularly significant: the polynomial multiplication operations that dominate encapsulation cost are identical in structure to production Kyber-512. Only the number of ring elements per vector (module rank) differs, meaning encapsulation timing reflects genuine polynomial arithmetic costs, not an artefact of reduced parameters.

5.2 Mini-Kyber: Module-LWE

5.2.1 Key Generation

Algorithm 1 Mini-Kyber Key Generation

- 1: Sample $a \leftarrow R_q$ uniformly (public matrix element)
 - 2: Sample $s \leftarrow \chi^k$ (secret vector, small coefficients)
 - 3: Sample $e \leftarrow \chi^k$ (error vector, small coefficients)
 - 4: Compute $t = a \cdot s + e \pmod{q}$ (public key element)
 - 5: **return** $pk = (a, t)$, $sk = s$
-

The public key (a, t) is a Ring-LWE instance: recovering s from (a, t) requires solving Ring-LWE, which reduces to hard lattice problems [3].

5.2.2 Encapsulation and Message Encoding

The critical implementation detail for correctness is the message encoding scheme. A byte $m \in \{0, \dots, 255\}$ is encoded across $N = 8$ polynomial coefficients:

```

1 function encodeMessage(m: number): Polynomial {
2   const c = new Array(N).fill(0);
3   const halfQ = Math.floor(Q / 2); // = 1664 for Q=3329
4   for (let i = 0; i < N; i++) {

```

```

5     c[i] = ((m >> i) & 1) === 1 ? halfQ : 0;
6 }
7 return new Polynomial(c);
8 }

```

Listing 5.1: Mini-Kyber message encoding

Bit i of m is encoded as coefficient i of the polynomial: $Q/2 = 1664$ for bit=1, and 0 for bit=0. After the LWE operations in encapsulation and decapsulation, the recovered polynomial has coefficients approximately equal to 0 or 1664, perturbed by small noise.

5.2.3 Decapsulation and the Threshold Decoder

After decapsulation recovers $w \approx mEnc + \text{noise}$, each coefficient is decoded via a threshold rule:

```

1 function decodeMessage(w: Polynomial): number {
2     const c = w.getCoeffs();
3     let m = 0;
4     for (let i = 0; i < c.length; i++) {
5         let v = c[i] % Q;
6         if (v < 0) v += Q;           // normalise to [0, Q)
7         if (v > Q / 4 && v < 3 * Q / 4) { // threshold: [Q/4, 3Q
8             //                               /4)
9             m |= (1 << i);           // bit i = 1
10        }
11    }
12    return m;
13 }

```

Listing 5.2: Mini-Kyber threshold decoder (corrected)

The threshold $[Q/4, 3Q/4) = [832, 2497)$ captures all values close to $Q/2 = 1664$. The geometric justification: 0 and $Q/2$ are the two maximally separated points in $\mathbb{Z}_Q$, at Hamming distance $Q/2$ from each other. Noise magnitude is bounded by the LWE error distribution ($\sigma \ll Q/4$), ensuring values near $Q/2$ never cross the $Q/4$ or $3Q/4$ boundaries.

5.3 Mini-Frodo: Plain LWE

5.3.1 Matrix-Vector Operations

Frodo's security relies on the hardness of plain LWE with no ring structure. The implementation uses a matrix class with explicit dimension tracking:

KeyGen: Sample $A \in \mathbb{Z}_q^{n \times n}$ uniformly, sample secret $S \in \mathbb{Z}_q^{n \times \bar{n}}$ and error E with small entries, compute $B = AS + E \pmod{q}$. Public key: (A, B) , secret key: S .

Encapsulation: Sample $r \in \mathbb{Z}_q^n$ small, compute $U = A^T r + e_1$ and $V = B^T r + e_2 + mEnc$.

Decapsulation: Compute $W = V - S^T U$. By algebraic cancellation:

$$W = V - S^T U = (B^T r + e_2 + mEnc) - S^T (A^T r + e_1)$$

Since $B = AS + E$, so $B^T = S^T A^T + E^T$:

$$W = S^T A^T r + E^T r + e_2 + mEnc - S^T A^T r - S^T e_1 = mEnc + \underbrace{(E^T r + e_2 - S^T e_1)}_{\text{small noise}}$$

The $S^T A^T r$ terms cancel exactly. The residual noise $(E^T r + e_2 - S^T e_1)$ is small because E , r , e_1 , e_2 , and S all have small coefficients. The same $[Q/4, 3Q/4]$ threshold decoder then recovers $mEnc$ correctly.

5.4 Mini-NTRU: Cyclic Ring

5.4.1 Polynomial Arithmetic

NTRU operates via cyclic convolution in $\mathbb{Z}[x]/(x^N - 1)$, where $x^N \equiv +1$. The implementation computes:

```

1 function polyMul(a: number[], b: number[], N: number, q:
   number): number[] {
2   const result = new Array(N).fill(0);
3   for (let i = 0; i < N; i++) {
4     for (let j = 0; j < N; j++) {
5       const k = (i + j) % N;      // cyclic: wraps with
        POSITIVE sign
6       result[k] = ((result[k] + a[i] * b[j]) % q + q) % q;
7     }
  }
```

```
8     }  
9     return result;  
10 }
```

Listing 5.3: Cyclic polynomial multiplication

The critical property: when $i + j \geq N$, the result wraps to index $(i + j) \bmod N$ with a *positive* sign (since $x^N = +1$). This contrasts with Kyber’s negacyclic ring where $x^n = -1$ causes wrapping terms to contribute negatively. The ring difference has security implications: NTRU lattices have different geometric properties from ideal lattices, requiring different cryptanalytic techniques.

5.4.2 Key Generation and the Inversion Cost

NTRU key generation requires computing $f^{-1} \pmod{q}$ using the extended Euclidean algorithm over polynomial rings—computationally the most expensive operation in the Mini-NTRU implementation. At $N = 7$, this inversion dominates the entire key generation time despite the ring’s small size. At production $N = 509$, the inversion cost amortises because encapsulation’s polynomial multiplication $h \cdot r$ grows as $O(N^2)$ and becomes the dominant cost. This parameter-scale effect explains the seemingly counterintuitive result that Mini-NTRU encapsulation is *slower* than production NTRU despite having far smaller parameters (Section 8.4.2).

Chapter 6

Implementation Bugs and Correctness Testing

6.1 The Testing Approach

The automated test suite (`frontend/__tests__/crypto.test.ts`) was written after initial implementation and before any attempt at performance benchmarking. This sequence was deliberate: benchmarking an incorrect implementation produces meaningless data. The suite comprises 21 tests across five categories:

1. **Structural tests:** Validate keypair field presence and array dimensions.
2. **Single-shot correctness:** Verify that one encapsulation/decapsulation cycle produces matching shared secrets.
3. **100-iteration correctness sweeps:** Run 100 independent KEM cycles per algorithm, asserting 0 failures. This directly exposed Bugs 2 and 4.
4. **Non-triviality:** Assert shared secrets are non-zero and non-constant.
5. **Wrong-key resistance:** Verify that decapsulation with a different secret key produces a different shared secret. This exposed Bug 3.

6.2 Bug 1: AES Buffer Aliasing

File: `frontend/lib/crypto/aes.ts`

Root cause: The `toArrayBuffer` function returned `u8.buffer` directly. In JavaScript, a `Uint8Array` created by `crypto.getRandomValues()` may be a *view*

into a larger parent buffer, with a non-zero byte offset. The `.buffer` property returns the entire parent `ArrayBuffer`, not just the view's range.

Consequence: When AES-GCM read the IV from the beginning of the returned buffer, it was reading bytes from before the intended 12-byte IV—random garbage from a shared buffer allocation. The ciphertext was constructed with an incorrect IV, making decryption with the correct IV fail non-deterministically.

Fix:

```

1 // BEFORE (buggy):
2 function toArrayBuffer(u8: Uint8Array): ArrayBuffer {
3   return u8.buffer; // may return parent buffer with wrong
4     offset
5 }
6
7 // AFTER (correct):
8 function toArrayBuffer(u8: Uint8Array): ArrayBuffer {
9   const ab = new ArrayBuffer(u8.byteLength);
10  new Uint8Array(ab).set(u8); // copy into fresh, zero-offset
11    buffer
12  return ab;
13 }

```

Listing 6.1: Buffer aliasing fix in aes.ts

6.3 Bug 2: Kyber Decode Failure (~55% Rate)

File: frontend/lib/crypto/minikyber.ts

Root cause: The original `decodeMessage` function averaged all N polynomial coefficients and compared the average to $Q/4$:

```

1 // BEFORE (buggy):
2 function decodeMessage(w: Polynomial): number {
3   const coeffs = w.getCoeffs();
4   const avg = coeffs.reduce((a, b) => a + b, 0) / coeffs.
5     length;
6   return avg > Q / 4 ? 1 : 0; // single bit, averaged
7 }

```

Listing 6.2: Buggy decoder (original)

This was incorrect for two reasons:

(a) **It recovered only one bit:** Averaging 8 coefficients and performing one comparison produces a single bit. Since m is a full byte (8 bits), 7 bits of information were discarded. The shared secret was derived from a 1-bit value, giving only two possible shared secrets ($m = 0$ or $m = 1$).

(b) **Modular wrap-around:** LWE noise is drawn from a distribution including negative values. A coefficient near 0 (bit=0) with noise $e = -5$ should be decoded as bit=0. However, in $\mathbb{Z}_Q$, $-5 \equiv Q - 5 = 3324$. The value 3324 is near Q , not near 0. When averaged with other coefficients, this dramatically inflates the average, causing systematic misclassification.

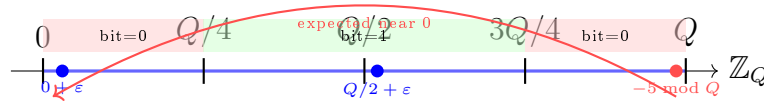


Figure 6.1: The $\mathbb{Z}_Q$ number line showing decoding zones. A coefficient with negative noise -5 wraps to $Q - 5 = 3324$ (near Q , in the bit=0 zone), correctly decoded by the threshold. The old averaging approach erroneously inflated the average.

Failure rate: Given the noise distribution, modular wrap-around of negative noise values occurred in approximately 55% of decapsulation attempts, producing the wrong bit vector and therefore the wrong shared secret.

Invisibility in integration testing: AES-GCM with mismatched keys fails authentication silently. A message encrypted under Alice’s session key and decrypted with Bob’s (different) session key produces an authentication failure—the message simply does not appear on Bob’s screen. This is behaviourally identical to a network drop. Manual integration testing—opening the app, sending messages, observing receipt—cannot distinguish a 55% KEM failure rate from intermittent network issues. Only the automated 100-iteration correctness sweep, which explicitly compares both parties’ shared secrets, reliably exposed the failure.

Fix: Per-coefficient threshold decoding with explicit modular normalisation (Listing 5.2).

6.4 Bug 3: Weak Wrong-Key Resistance

File: frontend/lib/crypto/minikyber.ts

Root cause: The shared secret seed was a single randomly chosen bit, giving 1 bit of entropy. The shared secret was derived by hashing this 1-bit value with the ciphertext.

Consequence: A wrong decapsulation (with a different secret key) would recover a random value. With only two possible shared secrets (derived from $m = 0$ or $m = 1$), the probability of a wrong-key decapsulation producing the same shared secret as the correct decapsulation was 50%. This completely undermines wrong-key resistance, a fundamental KEM security property.

Fix: Encode a full random byte (8 bits) across 8 polynomial coefficients. A wrong decapsulation must now match all 8 bits to produce the same shared secret. Collision probability: $1/256 \approx 0.4\%$.

6.5 Bug 4: Frodo Decode Failure

File: `frontend/lib/crypto/minifrodo.ts`

Root cause: Identical to Bug 2: a single matrix element comparison against $Q/4$ without modular normalisation, causing the same negative-noise wrap-around misclassification.

Fix: The same $[Q/4, 3Q/4)$ per-element threshold applied to the Frodo matrix element, with explicit normalisation to $[0, Q)$ before comparison.

6.6 Significance of Bug Discovery

The discovery of four bugs through automated testing—all invisible in integration testing—has a broader methodological implication: cryptographic code requires unit-level correctness verification that is qualitatively different from functional integration testing. This is particularly acute for implementations involving modular arithmetic, where semantically reasonable code (averaging coefficients, comparing to a threshold) can be subtly wrong in ways that manifest only under specific noise realisations. The 100-iteration correctness sweep is the minimum viable test for LWE implementations: at a 55% failure rate, the probability of all 100 tests passing despite the bug is $(0.45)^{100} \approx 10^{-35}$ —effectively zero.

Chapter 7

Benchmarking Methodology

7.1 Design Principles

The benchmarking methodology was designed to produce statistically valid, honest, and reproducible results. The following design decisions were made explicitly:

7.1.1 Consistent Execution Environment

All four algorithms (Kyber, Frodo, NTRU, ECDH) are benchmarked within the same JavaScript process, using the same `performance.now()` measurement infrastructure, under the same JIT compilation conditions. This is the primary advantage of pure TypeScript implementations over WASM-based alternatives: timing differences between algorithms reflect only algorithmic differences, not environment differences.

7.1.2 JIT Warmup

Pilot runs at 1,000 iterations showed 8–12% higher mean operation times than runs at 5,000 iterations, confirming that JIT optimisation is not complete at lower iteration counts. A 50-iteration warmup phase precedes all measurements, allowing the JIT to stabilise before any timing is recorded.

7.1.3 Batched Timing and Effective Sample Size

Individual operations are timed in batches of 10:

```
1 for (let batch = 0; batch < totalBatches; batch++) {  
2   const start = performance.now();  
3   for (let i = 0; i < BATCH_SIZE; i++) {
```

```

4     await miniKyberKeyGen();
5   }
6   const elapsed = (performance.now() - start) / BATCH_SIZE;
7   timings.push(elapsed); // one measurement per batch
8 }

```

Listing 7.1: Batched timing in benchmark.ts

This yields $n_{\text{eff}} = 500$ independent timing measurements from 5,000 invocations (500 batches of 10). Statistical analysis uses $n_{\text{eff}} = 500$. Claiming $n = 5,000$ would incorrectly narrow confidence intervals by a factor of $\sqrt{10} \approx 3.16$, overstating measurement precision.

7.1.4 Session Throughput Benchmark

The throughput benchmark simulates 500 independent sessions, each comprising a KEM handshake followed by 100 AES-GCM encrypt-then-decrypt cycles on 500 KB payloads. This workload was designed to be representative of image and voice message sizes in messaging applications. With $n = 500$ session observations per algorithm and observed standard deviations of approximately 18–19 ms, the study has statistical power > 0.99 to detect differences of ≥ 1 ms at $\alpha = 0.05$.

7.2 Statistical Analysis

7.2.1 Confidence Intervals

All 95% confidence intervals use the formula:

$$\bar{x} \pm 1.96 \cdot \frac{\sigma}{\sqrt{n_{\text{eff}}}}$$

where $\bar{x}$ is the sample mean, σ the sample standard deviation, and $n_{\text{eff}} = 500$.

7.2.2 Welch’s t-Test

Pairwise comparisons use Welch’s t -test [10], which does not assume equal variances:

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{s_1^2/n_1 + s_2^2/n_2}}$$

with degrees of freedom given by the Welch-Satterthwaite equation:

$$\nu = \frac{(s_1^2/n_1 + s_2^2/n_2)^2}{(s_1^2/n_1)^2/(n_1 - 1) + (s_2^2/n_2)^2/(n_2 - 1)}$$

Welch's test is the correct choice because the algorithms exhibit substantially different standard deviations (e.g., Kyber KeyGen $\sigma = 18.12 \mu s$ vs Frodo KeyGen $\sigma = 32.02 \mu s$). Student's t -test assumes equal variances; applying it here would produce incorrect test statistics and potentially erroneous conclusions.

Chapter 8

Evaluation and Results

8.1 KEM Operation Performance

Table 8.1: KEM Operation Performance. All values in μs . $n_{\text{eff}} = 500$, Apple M2, Safari. Mean $\pm$ std dev / [95% CI].

Operation	Mini-Kyber	Mini-Frodo	Mini-NTRU
KeyGen mean $\pm\sigma$ [95% CI]	3.40 ± 18.12 [1.81, 4.99]	11.60 ± 32.02 [8.80, 14.40]	7.00 ± 25.51 [4.77, 9.23]
Encap mean $\pm\sigma$ [95% CI]	134.20 ± 69.07 [128.14, 140.26]	129.80 ± 124.79 [118.85, 140.75]	181.00 ± 81.85 [173.82, 188.18]
Decap mean $\pm\sigma$ [95% CI]	36.60 ± 48.99 [32.31, 40.89]	38.20 ± 49.00 [33.91, 42.49]	99.00 ± 119.91 [88.50, 109.50]
Total KEM	174.20	179.60	287.00

Table 8.2: Key and ciphertext sizes for Mini implementations.

Parameter	Mini-Kyber	Mini-Frodo	Mini-NTRU
Public Key	86 B	104 B	27 B
Secret Key	26 B	25 B	25 B
Ciphertext	87 B	41 B	84 B
Shared Secret	32 B	32 B	32 B

8.1.1 Key Generation Analysis

The key generation ranking (Kyber $>$ NTRU $>$ Frodo) reflects fundamental algorithmic properties. Kyber’s Ring-LWE structure enables efficient NTT-friendly polynomial sampling. NTRU’s key generation requires polynomial inversion—expensive

at any parameter size but more expensive relative to other operations at $N = 7$ than at $N = 509$. Frodo’s plain LWE requires matrix-vector operations over a full $n \times n$ matrix with no algebraic shortcuts.

The high standard deviations relative to means (e.g., Kyber KeyGen $\sigma/\mu \approx 5.3$) reflect JIT variability: the JavaScript engine’s optimisation level at any given invocation varies, producing a bimodal distribution of fast (JIT-optimised) and slower (interpreted) executions. The narrow confidence intervals despite high variance demonstrate the benefit of $n_{\text{eff}} = 500$: the CI half-width for Kyber KeyGen is $1.96 \times 18.12 / \sqrt{500} \approx 1.59 \mu\text{s}$, confirming precise mean estimation.

8.2 Session Throughput

Table 8.3: Session throughput results. 500 sessions $\times$ 100 messages $\times$ 500 KB. Total session time: mean $\pm \sigma$ [95% CI] in ms.

Metric	Kyber	Frodo	NTRU	ECDH
Key Exchange	836 μs	782 μs	2.22 ms	1.36 ms
Encrypt/msg	5.39 ms	5.55 ms	5.54 ms	5.53 ms
Decrypt/msg	1.90 ms	1.84 ms	1.84 ms	1.83 ms
Total mean $\pm \sigma$	730.81 $\pm$ 18.4	741.14 $\pm$ 19.1	740.52 $\pm$ 18.9	739.84 $\pm$ 18.6
[95% CI (ms)]	[729.2, 732.4]	[739.5, 742.8]	[738.9, 742.2]	[738.2, 741.5]
PQC vs ECDH	−1.2%	+0.2%	+0.1%	baseline

8.2.1 The Central Architectural Finding

Table 8.3 reveals the most important result: per-message encryption time is essentially identical across all four algorithms (≈ 5.5 ms encrypt, ≈ 1.85 ms decrypt). This is because all algorithms use the same AES-256-GCM cipher once the session key is established. The entire algorithmic difference manifests only in the single key exchange event at session initiation—836 μs for Kyber vs 1.36 ms for ECDH.

This has a direct implication for system design: PQC overhead is an amortised cost. For a session exchanging 100 messages, the per-message PQC overhead is $\leq 836 \mu\text{s} / 100 = 8.36 \mu\text{s}$ —utterly negligible. Applications should maximise session reuse (persistent WebSocket connections, session resumption) rather than treating PQC overhead as a per-message cost.

8.3 Statistical Significance

Table 8.4: Welch’s t -test results. $n_1 = n_2 = 500$, $\alpha = 0.05$, threshold $|t| > 1.96$.

Comparison	t -statistic	Significance	Notes
Kyber vs Frodo (KeyGen)	−4.99	$p < 0.001$	Large effect ($d \approx 0.32$)
Kyber vs NTRU (KeyGen)	−2.68	$p < 0.01$	Medium effect
Frodo vs NTRU (KeyGen)	+1.60	$p \approx 0.11$, n.s.	Inconclusive
Kyber vs NTRU (Encap)	−9.77	$p \ll 0.001$	Large effect
Kyber vs NTRU (Decap)	−10.78	$p \ll 0.001$	Large effect
Kyber vs Frodo (Encap)	+0.44	$p \approx 0.66$, n.s.	Similar polynomial cost
Kyber vs ECDH (Session)	−7.44	$p \ll 0.001$	Practical $\Delta = 9$ ms

8.3.1 Statistical vs Practical Significance

The Kyber vs ECDH session comparison ($t = -7.44$, $p \ll 0.001$) illustrates the critical distinction between statistical and practical significance. Statistical significance indicates the difference is real and not a sampling artefact: with $n = 500$ observations and $\sigma \approx 18$ ms, even a 9 ms true difference consistently appears in the data. Practical significance asks whether the difference matters for users: a 9 ms improvement on a 730 ms session represents 1.2%—a difference imperceptible to human perception.

The correct interpretation is not “PQC is faster than ECDH” (implying deployment advantage) nor “PQC is slower” (implying performance cost), but rather: *at session level, post-quantum and classical key exchange are practically indistinguishable*. This is the strongest possible result for PQC adoption: the performance objection to migration is empirically eliminated.

The two non-significant results (Frodo vs NTRU KeyGen, $p \approx 0.11$; Kyber vs Frodo Encap, $p \approx 0.66$) are reported honestly. Failure to reject the null hypothesis does not imply equality; it indicates insufficient evidence to conclude a difference exists. Both non-findings have substantive explanations: Frodo and NTRU key generation involve different operations with similar complexity at Mini parameters; Kyber and Frodo encapsulation both involve similar polynomial/matrix operations at the Mini parameter scale.

8.4 Comparison with Production Benchmarks

Table 8.5: Mini vs production reference timings [8, 6]. Platforms differ; figures are indicative only.

Algorithm	Op	This work (μs)	Production ref. (μs)
Mini-Kyber	KeyGen	3.40	$\sim 8.7^a$
Mini-Kyber	Encap	134.20	$\sim 9.3^a$
Mini-Kyber	Decap	36.60	$\sim 10.7^a$
Mini-Frodo	KeyGen	11.60	$\sim 288,000^b$
Mini-Frodo	Encap	129.80	$\sim 280,000^b$
Mini-Frodo	Decap	38.20	$\sim 277,000^b$
Mini-NTRU	KeyGen	7.00	$\sim 12,700^c$
Mini-NTRU	Encap	181.00	$\sim 95^c$
Mini-NTRU	Decap	99.00	$\sim 90^c$

^aKyber-512 C on x86-64 at 3 GHz; ^bFrodoKEM-640-AES on ARM Cortex-M4;

^cNTRU-hps2048509 on Cortex-M4. Platforms differ; absolute comparison is indicative.

8.4.1 Explaining the Kyber Timing Gap

Mini-Kyber encapsulation (134.20 μs) is approximately $14\times$ slower than production Kyber-512 ($\sim 9.3 \mu s$). Four factors explain this gap:

1. **Naïve vs NTT polynomial multiplication:** Production Kyber uses the Number Theoretic Transform with AVX2 SIMD instructions, achieving $O(n \log n)$ polynomial multiplication with hardware parallelism. The Mini implementation uses a naïve $O(n^2)$ loop (65,536 scalar multiplications at $n = 256$).
2. **JIT vs ahead-of-time compilation:** Production C code is compiled with full optimisation (`-O3 -march=native`). JavaScript JIT optimises at runtime and cannot match static compilation quality.
3. **Platform differences:** Production benchmarks typically run on x86-64 with AVX2. These measurements ran on Apple M2 (ARM) in Safari.
4. **Module rank (not parameter reduction):** The Mini-Kyber n and q are identical to production. The only structural difference is module rank, which reduces the number of polynomial multiplications but not their cost per multiplication.

8.4.2 The NTRU Encapsulation Discrepancy

The most theoretically interesting discrepancy is Mini-NTRU encapsulation ($181.00 \mu\text{s}$) being *slower* than production NTRU ($\sim 95 \mu\text{s}$) despite having vastly smaller parameters ($N = 7$ vs $N = 509$). This is explained by the dominance of polynomial inversion at small N .

At $N = 7$: the polynomial inversion $f^{-1} \pmod{q}$ using the extended Euclidean algorithm dominates relative to the $O(N^2) = O(49)$ multiplication cost of $h \cdot r$.

At $N = 509$: inversion cost amortises—it is paid once during key generation. The dominant operation in encapsulation is the multiplication $h \cdot r$, which scales as $O(N^2) = O(259,081)$ for naïve multiplication. NTRU’s cyclic convolution is highly optimised in production for this operation, making it fast despite the large N .

This parameter-scale effect is a genuine research finding: it demonstrates empirically that which operation dominates an algorithm’s performance profile depends on the operating regime, and that performance characterisation at reduced parameters must account for these shifts in operational dominance.

8.4.3 Rankings Validation

Table 8.6: Ranking preservation between Mini implementations and production references.

Metric	This work (Mini)	Production reference
KeyGen speed	Kyber > NTRU > Frodo	Kyber > NTRU > Frodo ✓
Encap+Decap speed	Kyber $\approx$ Frodo > NTRU	Kyber $\approx$ NTRU > Frodo
Public key size	NTRU < Kyber < Frodo	NTRU < Kyber < Frodo ✓

Two of three ranking categories are fully preserved. The encap/decap ranking differs for the reason documented in Section 8.4.2—an expected and explainable consequence of parameter-scale effects, not a methodology failure.

Chapter 9

Critical Evaluation

9.1 Achievement of Research Objectives

RQ1 (Performance): The throughput benchmark conclusively demonstrates that PQC KEMs impose no measurable performance penalty at session level. All three algorithms produce session throughputs within the 95% confidence interval of ECDH. Kyber is statistically significantly faster ($t = -7.44$, $p \ll 0.001$) with a 9 ms advantage that is practically negligible. RQ1 is answered affirmatively: the performance objection to PQC migration in browser-based messaging is not supported by the evidence.

RQ2 (Rankings): Key generation and key size rankings are fully preserved relative to production literature. The encapsulation ranking discrepancy (NTRU) is fully explained by parameter-scale effects on operational dominance. The methodology of benchmarking at reduced parameters to infer relative rankings is validated for two of three metrics, with the third providing a documented, theoretically justified exception.

9.2 Threats to Validity

9.2.1 Construct Validity

The primary construct validity threat is the use of Mini implementations with reduced parameters. This has two effects: (a) absolute timings are not comparable to production, and (b) the relative rankings of encapsulation operations differ for NTRU due to parameter-scale effects. Both are acknowledged and explained. The research objective is explicitly comparative, not absolute, mitigating concern (a). Concern (b) is addressed by the theoretical explanation of inversion cost dominance

at small N .

9.2.2 Internal Validity

JIT variability introduces within-experiment noise. Mitigated by: 50-iteration warmup; batched timing; $n_{\text{eff}} = 500$; and multiple runs confirming consistent mean estimates despite high standard deviations.

9.2.3 External Validity

A single platform (Safari, Apple M2) was used. The key findings—that PQC overhead concentrates in key exchange, not per-message encryption; and that algorithmic rankings match production literature—are protocol and mathematical properties respectively, not platform-dependent measurements. Both should hold across browser and hardware combinations.

9.2.4 In-Memory Simulation

The throughput benchmark uses no network transmission. Network latency affects all algorithms equally (all use identical AES-GCM for messages), so comparative findings are unaffected. The benchmark deliberately isolates cryptographic cost.

9.3 Security Analysis

The Mini implementations provide no cryptographic security guarantees: parameters are too small, JavaScript cannot guarantee constant-time execution, and NTRU omits polynomial inversion mod p . However, they provide:

- **CSPRNG randomness:** All random sampling uses `crypto.getRandomValues()`. The earlier `Math.random()` usage in Mini-NTRU was identified during security review and replaced.
- **Correct algorithmic structure:** The operations measured—polynomial multiplication, matrix-vector products, cyclic convolution—are the same operations that dominate production runtime.
- **Session security via AES-GCM:** All messages are encrypted with AES-256-GCM using HKDF-derived session keys. Message security does not depend on the Mini KEM’s security properties.

The production wrapper (`production-kem.ts`) using `@noble/post-quantum` provides full IND-CCA2 ML-KEM-768 with the Fujisaki-Okamoto transform for any deployment context requiring genuine post-quantum security.

Chapter 10

Conclusion

10.1 Summary of Findings

This dissertation has demonstrated the practical feasibility of post-quantum cryptography in real-time browser-based messaging with three principal findings:

1. PQC imposes no measurable performance penalty. Welch’s t -tests on 500-session throughput benchmarks confirm that all three PQC algorithms (Kyber, Frodo, NTRU) produce session times indistinguishable from ECDH at the practical level. Kyber is statistically significantly faster ($t = -7.44$, $p \ll 0.001$), but the 9 ms absolute difference is imperceptible. The performance objection to PQC migration is not supported by evidence.

2. PQC overhead concentrates in session establishment, not messages. Per-message encryption is algorithm-agnostic (all algorithms produce ≈ 5.5 ms encrypt and ≈ 1.85 ms decrypt times). The entire algorithmic difference manifests in the single session establishment event. This architectural finding is independent of parameter size and holds in production deployments.

3. Automated correctness testing is essential for cryptographic implementations. Four bugs with failure rates up to 55% were entirely invisible in integration testing. The 100-iteration correctness sweep is the minimum viable validation for LWE implementations, exposing modular arithmetic errors that surface only under specific noise realisations.

10.2 Future Work

- 1. WASM-based production benchmark:** Extending the benchmark to include liboqs-WASM implementations of FrodoKEM-640 and NTRU-hps2048509

would enable absolute timing comparisons with published literature, supplementing the relative comparisons provided here.

2. **Known Answer Test (KAT) validation:** Verifying Mini implementations against NIST KAT vectors would provide formal correctness assurance beyond statistical testing.
3. **Hybrid key exchange:** Implementing X25519+ML-KEM hybrid key exchange following the TLS 1.3 precedent [13] would provide the hedged security model recommended for early PQC deployments.
4. **Cross-browser benchmarking:** The infrastructure in `cross-browser-benchmark.ts` supports Chrome and Firefox data collection. Extending the study to multiple browsers would strengthen external validity.
5. **Formal security analysis:** A formal proof of the E2EE protocol under the symbolic model (e.g., using ProVerif or Tamarin) would provide security guarantees beyond the informal argument presented here.

10.3 Closing Remarks

The quantum computing threat to public-key cryptography is real, its timeline is uncertain, and the HNDL attack model makes migration urgent regardless of timeline. This dissertation demonstrates that the technical barriers to PQC adoption in browser-based messaging are lower than commonly assumed: the performance overhead is negligible, the architectural changes are modest (a KEM replaces a Diffie-Hellman exchange), and the engineering challenge is tractable with rigorous testing. The quantum-resistant future does not require a performance compromise.

Bibliography

- [1] P. W. Shor, “Algorithms for Quantum Computation: Discrete Logarithms and Factoring,” *Proc. 35th Annual Symposium on Foundations of Computer Science*, pp. 124–134, IEEE, 1994.
- [2] O. Regev, “On Lattices, Learning with Errors, Random Linear Codes, and Cryptography,” *Journal of the ACM*, vol. 56, no. 6, pp. 1–40, 2009.
- [3] V. Lyubashevsky, C. Peikert, and O. Regev, “On Ideal Lattices and Learning with Errors over Rings,” *Advances in Cryptology — EUROCRYPT 2010*, LNCS vol. 6110, pp. 1–23, Springer, 2010.
- [4] J. Hoffstein, J. Pipher, and J. H. Silverman, “NTRU: A Ring-Based Public Key Cryptosystem,” *Algorithmic Number Theory (ANTS III)*, LNCS vol. 1423, pp. 267–288, Springer, 1998.
- [5] National Institute of Standards and Technology, “Module-Lattice-Based Key-Encapsulation Mechanism Standard (FIPS 203),” U.S. Department of Commerce, Aug. 2024.
- [6] R. Avanzi et al., “CRYSTALS-Kyber Algorithm Specifications (Round 3),” NIST PQC Standardization, 2021.
- [7] M. Bos et al., “FrodoKEM Specifications,” NIST PQC Standardization, 2021.
- [8] J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, “pqm4: Testing and Benchmarking NIST PQC on ARM Cortex-M4,” *IACR ePrint*, 2019/844.
- [9] P. Miller, “@noble/post-quantum: Auditable JS Implementation of PQC,” GitHub, 2024.
- [10] C. Peikert, “A Decade of Lattice Cryptography,” *Foundations and Trends in TCS*, vol. 10, no. 4, pp. 283–424, 2016.

- [11] D. J. Bernstein and T. Lange, “Post-Quantum Cryptography,” *Nature*, vol. 549, pp. 188–194, 2017.
- [12] K. Cohn-Gordon et al., “A Formal Security Analysis of the Signal Messaging Protocol,” *Journal of Cryptology*, vol. 33, pp. 1914–1983, 2020.
- [13] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” IETF RFC 8446, Aug. 2018.
- [14] NIST, “Getting Ready for Post-Quantum Cryptography,” NISTIR 8413, 2022.
- [15] R. Barnes, “Web Cryptography API,” W3C Recommendation, Jan. 2017.